SOEN 387 — Assignment 2

Seat Holds

Domain layer, concurrency, and persistence · Chapters 5–7 · ~2 weeks · individual

This assignment builds on your Assignment 1 Encore (or the provided reference solution). You will replace "reserve instantly" with a **two-step hold**, give seats a real lifecycle, make state survive a restart, and — the heart of the assignment — keep the whole thing **correct under concurrency**. This is where the course stops being about one request and starts being about shared, contended state.

The feature: hold, then confirm

Booking becomes two steps. A caller first **holds** a seat, which reserves it *provisionally* for a fixed window (say 90 seconds — a configurable **hold TTL**). Within that window they **confirm** the hold, turning it into a real reservation. If they never confirm, the hold **expires** and the seat returns to the pool automatically.

A seat is therefore occupied by **a confirmed reservation OR a live hold** — and this single rule is what makes "seats left" correct. If you did Assignment 1's waitlist, an expiring or cancelled hold is the natural moment a waitlisted person could be offered the seat (promotion remains optional here).

What you will build

1 · A domain model with a lifecycle (Chapter 5)

- Model the reservation as a **state machine**: held → confirmed | released | expired. Put the transitions and the invariant in the **domain layer**, not in a handler.
- The invariant — **confirmed reservations + live holds never exceed capacity** — is enforced at the only doors through which it could be violated (hold, confirm). You must not be able to construct an over-booked event.
- seats_left is a computed fact (capacity minus confirmed minus live holds), stated once. An expired hold frees its seat *by rule* — not by a cleanup job.
- Keep the domain module's import list empty: no Flask, no SQL, no HTTP.

2 · Contract (Chapters 2–3)

```
POST   /events/{eventId}/holds             hold a seat (Idempotency-Key required)
POST   /holds/{holdId}/confirm             confirm a hold -> a reservation
DELETE /holds/{holdId}                      release a hold early
GET    /holds/{holdId}                       hold status + secondsRemaining
```

- Hold creation is idempotent (same key -> same hold). Confirm is idempotent by meaning (confirming a confirmed hold is the same end state).
- Add coded errors: sold_out (no seat to hold), hold_expired (confirm after the TTL), hold_not_found, already_confirmed. Document retryability.
- Keep it backward compatible where it can be, and version-boundary anything that cannot. State your policy.

3 · Persistence (Chapter 6)

- Move state into a real database (SQLite is fine; PostgreSQL recommended). State must **survive a restart**: hold a seat, kill the process, restart, and the hold (and its remaining time) is still there.
- Provide at least **two migrations** (e.g. the initial schema, then the holds table) applied by a small, ordered migration runner — not hand-run ALTERs.
- Put persistence behind a repository interface so the domain and service layers never learn what a database is (the in-memory version becomes your test double).

4 · Concurrency — the core (Chapter 7)

Two callers must never hold the same last seat. Prevent it with a real concurrency-control discipline:

- Choose **optimistic** (a version column + conditional write + bounded retry) or **pessimistic** (SELECT … FOR UPDATE within a transaction). Implement it, and **justify the choice** in your design note.
- Write a **concurrency test**: launch N concurrent hold requests against an event with 2 seats and show that exactly 2 succeed and the rest get sold_out — with **no oversell**, reproducibly. Include the script and its output.
- Explain, in one paragraph, the race you are preventing (the interleaved read-check-write) and why your mechanism closes it.

5 · Expiry

- Holds expire on their own. Expiry must be correct **without** relying on a background sweeper having run: seats_left must be right the instant a hold's TTL passes, because the rule (not a cron job) defines it. A sweeper that tidies expired rows is allowed, but it is an optimization, not the source of truth.

Deliverables

1. **The extended contract** (holds + confirm + errors).
2. **The implementation**: a clean domain layer (state machine + invariant), a service layer, a repository, and the database with its migrations.
3. **A concurrency test** that demonstrates no oversell under load, with its output.
4. **A restart demonstration**: evidence that a hold survives a process restart.
5. **The conformance suite**, passing (old + new).
6. **A design note (2 pages)**: the state machine; where the invariant is enforced; optimistic vs. pessimistic and why; how expiry stays correct without a sweeper.
7. **An AI-collaboration log** (see below).

Grading (100 pts)

Dimension	Pts	What full marks look like
Concurrency correctness	30	no oversell under concurrent load; a convincing, reproducible test; a correct account of the race and the fix
Domain model	25	clean state machine; the invariant enforced at the only doors; seats_left computed once; empty domain imports
Persistence	20	state survives restart; ≥ 2 ordered migrations; a repository seam; domain unaware of the DB
Contract & errors	15	holds/confirm modeled well; idempotency on hold and confirm; coded errors with retryability
Expiry & design note	10	expiry correct without a sweeper; the note defends the concurrency choice

Automatic cap: any demonstrable oversell (confirmed + held > capacity) caps the assignment — it is the one failure this assignment exists to prevent.

Hints

- Start from the domain layer with *no database* — get the state machine and invariant right in memory, unit-test them, then add persistence behind the repository. This is the Chapter 5 → 6 order, and it keeps concurrency reasoning clean.

- For the concurrency test, add a small artificial delay between read and write in your handler to *widen the race window* — it makes an unsafe implementation fail reliably, so you can prove your safe one holds.
- SQLite serializes writes and can hide a race; if you use it, test concurrency against the mechanism you'd actually deploy, and say so. PostgreSQL gives you real row locks.

Using AI

Use AI freely to scaffold the state machine, the migrations, and the tests — but **concurrency is exactly where confident AI code is most often subtly wrong**. You must understand your locking/versioning well enough to explain, at an oral spot-check, why two threads cannot both take the last seat. Keep an **AI-collaboration log**: include at least two concurrency- or persistence-related points where the assistant's first answer was wrong or incomplete and you fixed it.